

Automatic Adaptation of Android Applications for Run-time Permissions

With the introduction of new android runtime permission mechanism from Android 6.0 (Marshmallow) android developers are required to open up their old android application's source code and revamp according to the permission requirement which is time consuming and sometimes impossible. This paper propose a novel technique that detects the compatibility issues in a given app and fixes them when found, hence automatically adapting the app to the run-time permission mechanism.

Additional Key Words and Phrases: Android run time permission mechanism, Call graph, Android application instrumentation, Android system permission

ACM Reference Format:

. 2017. Automatic Adaptation of Android Applications for Run-time Permissions . 1, 1 (November 2017), 4 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

Android permission mechanism plays a major role on securing highly sensitive data of the android system. Prior to Android-6.0 (Marshmallow), the user grants the required permission at install time and application will be able to access all the granted resources without any further acknowledgement during the runtime. This creates permission related security concerns to the android users. After Android-6.0 (Marshmallow), user is privileged to revoke granted permission any time after application installation. Android applications must request run time permission when the application need access to resources where system permission is required.

Because of this, all the applications which were developed targeting earlier versions of Android SDK must be revamped to facilitate the new android runtime permission mechanism. Updating individual application to be compatible with new runtime permission is time consuming. This might even be impossible due to the unavailability of source code. Hence we propose to develop a tool - APKDroid technique that detects the compatibility issues in a given app and fixes them when found, hence automatically adapting the app to the run-time permission mechanism. The market share of new apps has grown significantly (over 10%) in less than a year, suggesting an urgent need for such an automatic adaptation.

1.1 Problem Statement

Android system permissions are divided into two categories, namely Normal and Dangerous. System automatically grants permission to the normal group whereas user permission is required for the dangerous group. Android application developers must define the required permissions on the application manifest file. Since the new android-M runtime permission mechanism, user can grant permission when the application is running and user can revoke granted permission at anytime. Further, user can give set of permission to an application while restricting another set of permissions. For an example, a user could choose to give permission to a messenger

Author's address:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, or post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2017 Association for Computing Machinery.

XXXX-XXXX/2017/11-ART \$15.00

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

application to access the photo gallery but not the device location.

If the application tries to call a function that requires user permission which has not yet been granted, the function will suddenly throw an Exception which will lead to crash the application. Hence developer must make sure to check and request required permission before invoking a permission required method. This new permission mechanism will work as described for the android application which has targetSdkVersion as 23 or higher when it is running on a device with android version 6 or higher. The same application will work for old android version with the old behavior.

When the application is running on a device with android version 6 or higher with targetSdkVersion less than 23 application might crash. For this setting the device will switch to the old behavior and it will ask to provide permission at the installation time. But the problem is, still the user can revoke the permission after the application installation. In such case, if permission required method is called, it will return NULL or zero instead of the exception. If the application tries to access the above returned values, then the application will be crashed. Even though some actions have been taken to accommodate legacy applications (targetSdkVersion less than 23) in Android-M, such as not to fire exception when accessing permission required method, it is much needed to update the legacy applications to maintain its proper behavior.

Even though developers can code their new application with the requirements of new runtime permission mechanism, developers may have to reopen their old code and revamp the code to check the user granted permissions. This could be impossible due to many practical reasons. Therefore a technique to analyse and change android byte code is essentially required which make developer's life easier.

The rest of the paper is organized as follows. In Section ??, ...is presented in Section ?. In Section ??, ...In Section ??,... Section ? concludes the paper with remarks.

2 METHODOLOGY

Android applications which has targetSdkVersion less than 23 will be modified to request required permission when invoking permission required methods. Manifest file of the app will be extracted to get initial required information such as targetSdkVersion and required permissions. Since developers must define the required permission at the manifest file, we can assume all the required permission to run the application on android are defined in the manifest file. If dangerous permissions are found in the application's manifest file and the targetSdkVersion is less than 23, this application must be processed and updated.

Android system method with its required permission must be used to find the places where those methods are used. Finding usage of permission required method is done in two ways. Analyzing the call graph to find out usage of permission required system methods is a direct approach. As the next approach, we analyzed signatures of methods in call graph to find out usage of permission required class objects. If there are such arguments, we considered that as a permission required method.

Then next step is to identify callers in the android application which invoke the above android system method. Application's call graph must be constructed to find the callers of the android permission needed system methods. Then, such callers can be found by traversing upward through the call graph. Call graph gives a tree of source methods and target methods. The requirement is to traverse upward through the constructed call graph and find

ALGORITHM 1: Finding lowest level caller of the permission required android system methods**Input:** *InputList* - list of permission required android system methods**Output:** *OutputList* -List of callers of input methods in the application*Index* = 0;**repeat** $permissionNeededMethod = InputList[Index];$ $traverseUpward(permissionNeededMethod);$ $Index ++;$ **until** $Index < length\ of\ InputList;$ **Function** $traverseUpward(permissionNeededMethod)$ $SourceList =$ List of sources of the $permissionNeededMethod$ obtained from call graph; $index = 0;$ **if** ($permissionNeededMethod$ is application method) **then** $OutputList[Index] = permissionNeededMethod;$ /* lowest caller found;

*/

return; **end** **for each of** $SourceList$ **do** **if** ($SourceList[index]$ is visited) **then** **continue;** **else** $traverseUpward(SourceList[index]);$ **end** $index++;$ **end**

the lowest possible caller in the android application (android developer's code).As Algorithm 1 demonstrates, the way of finding modification needed methods in the android application. Recursively traversing through the call graph will be much more effective way when traversing upward through the call graph. The modification point must be a caller of permission needed android system method as well as the point must be in android developer's code (It could be an over-ridden callback or a method introduced by the developer). We travel upward until we reach the point where both the conditions are met and if found that would be the lowest caller of the method in the application.

After finding the places where modifications are needed in order to adapt for new runtime permission mechanism, the next step is to modify them appropriately. To have a stable well adapted applications we need to deal with unpredicted nature of the application's code. The basic idea is to insert required code (or repair) pieces into the android applications for automatic adaptation.The basic functionality of inserting code is to check the required permissions are granted and if not request required permissions. Updating application's code might be tricky due to unpredictable nature of the code. Some methods may require more than one permission to be processed. For such scenario we need to check multi permissions and request only non-granted permissions. Some applications may have developed the permission request mechanism partially. Therefore, we need to make sure following two conditions are met at the end of processing.

- All the required permissions for each and every methods are checked before accessing
- All the non granted required permission are requested before invoking the method

The transformation would start with an inspection of existing handling logic related to run-time permission, followed by either code insertion (if run-time permission handling is entirely missing) or repair (if the handling is incomplete or broken).

Figure 1 shows an overall process of making automatic adaptation to new runtime permission mechanism starting from legacy android application to create well adapted application.

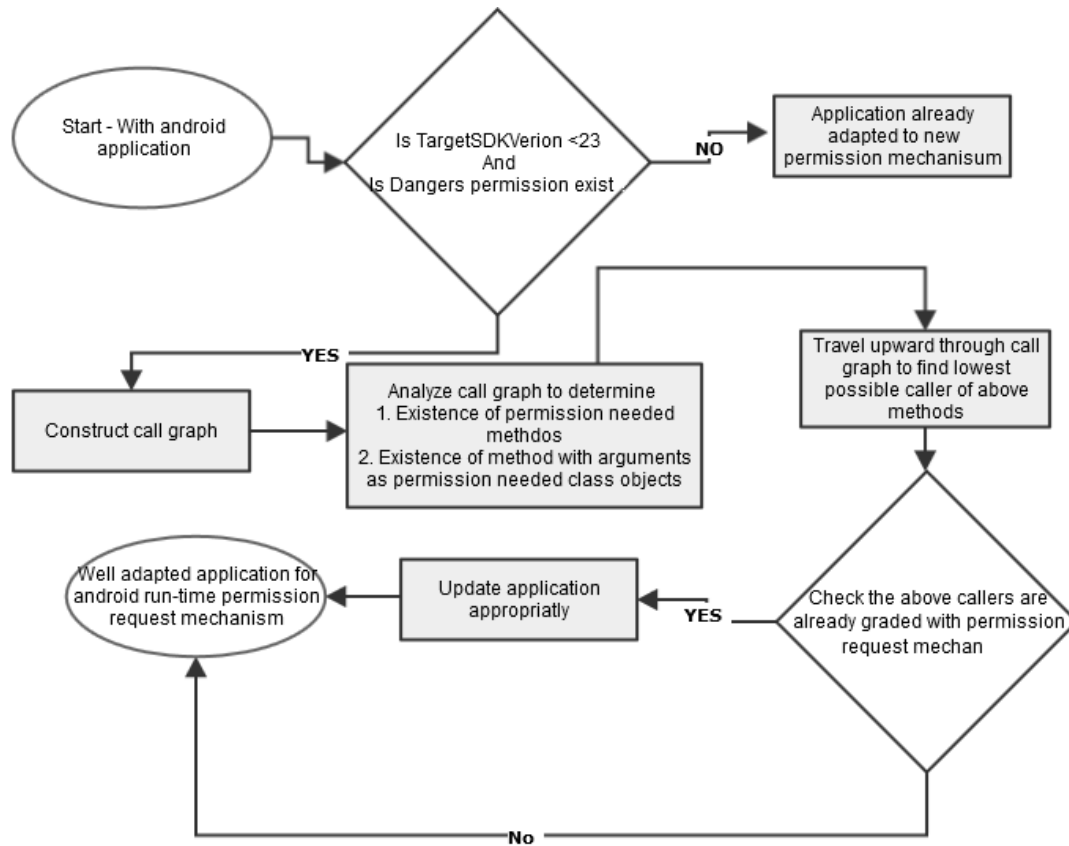


Fig. 1. Overall process

3 TOOL

4 RESULTS

5 CONCLUSION

ACKNOWLEDGMENTS

REFERENCES